

Writing the Requirements Portion of a Feature Spec

A student guide for drafting the **beginning** of a feature specification: header, user stories, functional requirements, **initial data model**, and acceptance criteria (AC).

This is the **what**, **what data exists**, and **how we know it's done** part of the file. Save API contracts and screen layouts for after these are clear. Full section order and process live in [framework.md](#). Canonical example: [feature-1-user-auth.md](#).

File name: `features/feature-N-short-name.md` (e.g. `feature-2-todo-list-management.md`)

Status while writing: leave `Draft` until stories, FRs, initial data model, and Gherkin are complete and consistent; then set `Ready` .

Feature vs user story

A **feature** is a shippable slice of the product: one capability area that can live in its own Git branch, merge to `dev` , and leave the app usable. One feature file (`feature-N-*.md`) owns that whole slice.

A **user story** is one outcome *inside* that feature — something a person (or the application) can do — with its own priority, independent test, and Gherkin scenarios. Stories are **not** separate files and **not** separate branches.

	Feature	User story
Grain	A product capability / vertical slice	One user (or system) outcome inside that slice
Artifact	One file: <code>features/feature-N-short-name.md</code>	Sections <code>US-N.n</code> inside that file
Git	One branch: <code>feature/N-short-name</code>	No branch of its own
Size check	Often days of work; several stories; shared data/API/screens	Verifiable in one independent test
Question it answers	"What capability are we adding to the product?"	"What can someone <i>do</i> once this capability exists?"

Rule of thumb: if you would open a new branch and merge a coherent chunk of app behavior, it is a **feature**. If it is one of several related outcomes that share the same screens, tables, and release, it is a **story**.

Features in this project (examples)

Feature	Why it is a feature (not a story)
1 — User Authentication & Session Management	Whole identity boundary: register, session, protect routes. Foundation for every later feature.
2 — Todo List Management	Users own lists as a first-class thing (create/view/rename/delete + privacy).
3 — Todo List Item Management	Users own items inside lists (add/complete/edit/delete). Distinct entity and UI from Feature 2.
4 — User Profile Management	Profile view/edit and logout placement — account UX, not list/todo CRUD.
5 — Todo Due Date	A focused delta on existing todos (optional <code>dueDate</code> + overdue display). Still its own feature because it has its own FRs, AC, and branch.

Stories inside a feature (examples)

Feature 1 is one capability (**auth**). Its stories are the outcomes that make up that capability:

Story	Outcome (not a separate feature)
US-1.1 Register an account	Create credentials
US-1.2 Sign in	Start a session
US-1.3 Stay signed in across page loads	Persist session in the browser
US-1.4 Sign out	End session
US-1.5 Block unauthenticated access	Guard protected routes/API

Feature 2 is **list management**; stories carve that into create / view / manage rows / rename-delete / privacy — all one feature, five stories.

Feature 5 is smaller but still a feature: the *capability* is “due dates on todos.” Stories are set / view / edit-clear / overdue highlighting — not four separate features.

Common mistakes

Mistake	Better split
One “mega feature” for the whole Todo app	Split like this repo: auth → lists → items → profile / due date
One feature file per tiny UI tweak with no shared release intent	Often a story (or a small delta feature like Feature 5 if it has its own AC/branch)
Treating “Sign in” as Feature 1 and “Register” as Feature 2	Same capability area → one feature, two stories (as in Feature 1)
Putting list CRUD and todo-item CRUD in one feature	Separate features when entities, APIs, and screens diverge (Features 2 vs 3)

Dependencies: features may depend on earlier features (Depends on:). Stories within a feature share that feature's branch and ship together.

Naming the feature

The feature name appears in three places that must stay aligned:

Place	Form	Example
Spec title (# Feature: ...)	Title Case, human-readable	Todo List Management
File name	feature-N-short-name.md	feature-2-todo-list-management.md
Git branch	feature/N-short-name	feature/2-todo-list-management

The **short-name** is a kebab-case slug of the title (no Feature ID words like `feature` inside the slug). Epic titles in Agility export come from `# Feature: ...`, so name it carefully.

Principles

1. **Name the capability, not a single action.** Prefer a noun phrase for the product area ("User Authentication", "Todo List Management") over one verb story ("Sign In", "Create List").
2. **Be specific enough to tell features apart.** "Lists" vs "List Items" matters — this repo uses **Todo List Management** (Feature 2) and **Todo List Item Management** (Feature 3) on purpose.
3. **Keep it short (about 2–5 words).** Long enough to be clear; short enough for a branch name. Drop filler ("The Amazing New...", "Module", "System", "Implementation").
4. **Use product language, not stack language.** Good: `User Profile Management` . Bad: `Sequelize User Controller / Vuetify MenuBar Dropdown` .
5. **Prefer durable domain terms.** Name what the product *is* about, not a temporary UI widget. Good: `Todo Due Date` . Weaker: `Orange Date Picker On Dashboard` .
6. **Match title ↔ file ↔ branch.** Same words, same order. If you rename the title, rename the file and branch pattern (and update the catalog).
7. **Do not put story verbs in the feature title.** Story titles carry the verbs (`Register an account` , `Sign in`). The feature title names the whole area those stories belong to.
8. **Delta features still get a clear capability name.** Feature 5 is `Todo Due Date` — not `Update Feature 3` or `Misc Todo Tweaks` .

Good vs weak names (this project)

Prefer	Avoid	Why
User Authentication & Session Management	Login Page	Capability vs one screen/story
Todo List Management	CRUD for lists	Product term, not tech jargon
Todo List Item Management	Todos	Too vague — lists vs items collide
User Profile Management	MenuBar Changes	Names the capability, not the widget
Todo Due Date	Add dueDate Column	Product outcome, not a schema ticket

Checklist before you freeze the name

- ☐ A teammate can guess the scope from the title alone
- ☐ It does not sound like a single user story
- ☐ kebab-case short-name is readable in `feature/N-short-name`
- ☐ It will not be confused with another planned feature's name

What you are authoring

Piece	Answers	ID style
User stories	Who needs what, and why?	US-N.n
Functional requirements	What must the system do (rules)?	FR-00N
Initial data model	What things exist, and what fields/relationships does this feature need?	Key Entities → table sketches
Acceptance criteria	How do we prove a story works?	Gherkin under ### US-N.n

Stories describe **outcomes for a person** (or for the system as actor).

FRs are **testable system rules** (MUST / MUST NOT).

The **initial data model** names the entities and the fields this feature introduces or changes.

AC scenarios are **concrete examples** that map 1:1 to automated tests later.

Do **not** invent stack or file paths in stories. Prefer product language ("sign in", "own lists") over implementation ("call Sequelize", "use axios").

1. Start with the header

```
# Feature: <Human-readable title>
```

```
**Feature ID:** N
```

```
**Branch pattern:** `feature/N-short-name`
```

```
**Status:** Draft
```

```
**Created:** YYYY-MM-DD
```

```
**Input:** One sentence – what this feature is for (intent, not technology)
```

```
**Depends on:** [Feature X – ...](feature-X-...md) ← omit if none
```

```
**Related:** optional links to ADRs or reference docs
```

Tips

- **Feature title / short-name** — follow [Naming the feature](#); keep `# Feature: ...`, file name, and branch pattern aligned.
- **Feature ID** — next unused integer in the catalog; never reuse a retired ID.
- **Branch pattern** — must match the Git branch you will use (`feature/N-short-name`).
- **Input** — the problem or capability in plain language (the "why we are writing this").
- **Depends on** — only features that must already be merged to `dev` .

2. Write user stories

What a user story is

A **user story** is a short, testable statement of value from a stakeholder's point of view. In this kit it lives **inside** a feature file — not as its own file or branch.

It answers three questions:

Line	Question	Example (US-1.2)
As a ...	<i>Who</i> cares?	registered user
I want to ...	<i>What</i> can they do?	sign in with username and password
So that ...	<i>Why</i> does it matter?	access the dashboard securely

Stories describe **outcomes**, not implementation. They drive FRs (rules), Gherkin AC (proof), and later tests. If you cannot name an **Independent test** for the story alone, the story is too vague or too big.

Also in every story block (this kit):

Field	Purpose
-------	---------

Short title	Human label (Sign in) — used in headings and maps
US-N.n ID	Traceability to AC and Test Coverage Map
Priority	P1 / P2 / P3 for what must ship in this feature
Independent test	One-sentence way to verify this story without the rest unfinished
Acceptance scenarios	Pointer to the matching ### US-N.n Gherkin block

List every meaningful outcome this feature delivers. Number them **US-`<feature-id>.<story-number>`** (restart at `.1` in each file).

Principles for writing user stories

1. **One outcome per story.** If you need “and” to glue two capabilities (“register **and** create a list”), split them — or put them in different features when the domains differ.
2. **Lead with the user (or the application).** Prefer a specific role (`new user` , `signed-in user`) over generic “user.” Use **As the** application for guards (redirects, `401` , blocking cross-user access).
3. **“I want” is a capability, not a UI widget.** Good: “rename my list.” Weak: “click the pencil icon in `v-data-table` .”
4. **“So that” must add real value.** It should not restate the “I want.” Good: “so that I can organize work separately.” Weak: “so that I can rename my list.”
5. **Keep the story thin; put rules in FRs and examples in AC.** The story states the outcome. Password length, exact error strings, and HTTP codes belong in FRs / Gherkin — not stuffed into the “I want” line.
6. **Make it independently testable.** The **Independent test** line should be something you could try in one pass (API or UI) even if sibling stories are incomplete.
7. **Prioritize honestly.** `P1` = must ship with this feature; `P2` = important; `P3` = nice-to-have. Do not mark everything `P1`.
8. **Align the title with the AC heading.** Story `US-2.1: Create todo lists` ↔ `### US-2.1 – Create todo lists` under Acceptance Criteria.
9. **Stay inside the feature’s capability.** Feature 2 stories are about **lists**; do not sneak in “complete a todo” (that is Feature 3).
10. **Write for a teammate and for tests.** A developer should know what “done” means; a tester should know what to exercise. Ambiguous stories produce ambiguous code.

Template

```
## User Stories

### US-N.1: Short title
**As a** <role>
**I want to** <capability>
**So that** <benefit>

**Priority:** P1
**Independent test:** <how to verify this story alone, in one sentence>
**Acceptance scenarios:** see ### US-N.1 under Acceptance Criteria
```

Roles you will use often

- **As a** new / registered / signed-in user — person using the product
- **As the** application — security, redirects, blocking bad access

Do / don't (quick reference)

Do	Don't
One capability per story	Bundle register + lists + due dates into one story
Name a real role or the application	Vague "As a user I want everything to work"
Make So that a real benefit	Repeat the "I want" in different words
Set Priority and Independent test	Leave them blank or "test later"
Product language	Stack language ("call the login endpoint with axios")

Example (Feature 1 style)

```
### US-1.2: Sign in
**As a** registered user
**I want to** sign in with my username and password
**So that** I can access the application dashboard securely

**Priority:** P1
**Independent test:** Sign in with known credentials and receive session token + redirect to home
**Acceptance scenarios:** see ### US-1.2 under Acceptance Criteria
```

More examples from this project

Story	Why it works
US-2.1 Create todo lists	One clear outcome; lists only
US-2.5 Private lists only	System-as-actor style outcome (isolation), still one testable goal
US-3.3 Complete tasks	Verb + object matches a single behavior
US-5.4 Spot overdue todos	User-visible outcome, not "set CSS class on overdue rows"

Aim for **3–8 stories** for a typical feature. If you need more than ~10, the feature is probably too big — split it.

3. Write functional requirements (FR-00N)

After stories, state the **rules the system must obey**. These are not UI copy; they are constraints implementers and testers use.

Template

```
## Requirements

### Functional Requirements

— FR-001: System MUST ...
```

- ****FR-002****: Users MUST be able to ...
- ****FR-003****: ... MUST NOT ...

Restart numbering at **FR-001** in every feature file.

How to write a good FR

Do	Don't
Use MUST / MUST NOT (testable)	"Should preferably..." without a hard rule
One rule per FR	Paragraphs that mix five rules
Be specific enough to fail a test	"Auth must be secure" with no rule
Cover validation, ownership, and key behaviors from the stories	Duplicate the entire story text

Good: Users MUST authenticate with username + password (not email-only login).

Weak: Login should feel secure and fast.

While drafting, mark unknowns:

- ****FR-004****: Session lifetime MUST be [NEEDS CLARIFICATION: hours? days?]

Resolve every [NEEDS CLARIFICATION] before Status: Ready .

Bridge stories → FRs

For each story, ask: *What rules make this story true?* Those become FRs. One story often yields several FRs; one FR may support more than one story.

4. Add Assumptions, Edge Cases, and Success Criteria

Still part of the "requirements info" beginning — keep them short.

Assumptions

- What already exists (e.g. Feature 1 auth is on `dev`)
- What you are deliberately not building yet

Edge Cases

- Empty required field → ...
- Cross-user access → ...
- Duplicate / invalid input → ...

Success Criteria

- ****SC-001****: Every Gherkin scenario has at least one automated test before merge
- ****SC-002****: <measurable outcome for this feature>

SC-00N items are feature-local outcomes (restart at SC-001 each file). App-wide quality bars stay in docs/nfr/quality-attributes.md — do not paste that whole table into every feature.

5. Draft the initial data model

After FRs (and before or alongside Gherkin), capture **what data this feature needs**. Do this in two steps: conceptual entities first, then an initial table sketch.

Step A — Key Entities (conceptual)

Name the things in the domain and how they relate. **No column types here.**

Key Entities

- ****User****: registered account; owns lists and sessions
- ****Session****: server-side record tying a login token to a user
- ****List****: named group of todos; belongs to one user

Tips

- Only entities this feature **creates or changes** (plus parents you must reference, e.g. User when adding List).
- Say ownership in plain language (“belongs to one user”) — that later becomes FKs and isolation rules.
- If multi-user data is involved, also sketch **Data Ownership & Isolation** in one short bullet list (who can read/write what).

Step B — Data Model Requirements (initial tables)

Turn entities into **tables and fields** this feature will persist. This is still requirements — enough for implementers to build models — not a dump of Sequelize code.

Data Model Requirements

`users` table

Field	Type	Rules
`id`	INTEGER PK	Auto-increment
`username`	STRING(100)	Required, unique; stored lowercase
`password`	STRING(255)	Required; bcrypt hash only
...	...	...

`sessions` table

Field	Type	Rules
`id`	INTEGER PK	Auto-increment
`token`	STRING	Required
`userId`	INTEGER FK	Required, references `users.id`
...	...	...

How to write a good initial data model

Do	Don't
----	-------

One table section per new/changed entity	Invent tables for future features you are not building
Field, type, and rules (required, unique, FK, default)	Vague "user stuff in the DB"
Mark FKs and ownership (userId → users.id)	Forget who owns the row in a multi-user app
Align fields with FRs and AC (if AC mentions email, the table needs email)	Add columns "just in case" with no story/FR
Use [NEEDS CLARIFICATION: ...] for undecided types/lengths	Leave required fields unnamed

Delta features: if Feature N only adds a column (e.g. `dueDate` on `todos`), document **only the change** — new/changed fields — and point at existing tables in `features/reference/data-model.md` rather than rewriting the whole schema.

Check against stories: every piece of data a story creates, updates, or displays should appear as an entity/field (or be explicitly out of scope).

6. Write acceptance criteria (Gherkin)

What Gherkin AC is

Acceptance criteria are the proof that a user story is done. In this kit they are written as **Gherkin scenarios**: structured examples using **Given / When / Then** (and **And**).

Part	Role
### US-N.n – Title	Groups all scenarios for one story (must match the user story ID/title)
#### Scenario: ...	One concrete example (happy path or failure) — later becomes an <code>it("...")</code> name
Given	Preconditions / world state before the action
When	The single action under test
Then	Observable pass/fail outcomes
And	Extra preconditions or outcomes (same scenario)

Stories say *what* value we want. FRs say *rules*. **Gherkin says examples that must pass** — specific enough that a developer can implement and a test can assert them.

AC proves each user story with concrete scenarios. Group scenarios under a heading that uses the **same story ID** as the User Stories section.

Principles for writing Gherkin AC

- One story per ### block.** Do not mix US-1.1 and US-1.2 scenarios under one heading. Traceability depends on this.
- One behavior per scenario.** Prefer separate scenarios for "valid login" and "wrong password" over one mega-scenario with branches.
- Happy path first, then failures.** Lead with the success case; follow with validation, auth failures, and ownership/edge cases from your Edge Cases list.
- Name scenarios like test titles.** Use a clear, unique phrase (`User signs in with valid credentials`). That exact string should later appear in the Test Coverage Map and preferably in `it("...")` .

5. **Make Then observable.** Assert status codes, JSON fields, redirects, visible messages, or persisted data. Avoid “the system works” or “the user is happy.”
6. **Quote exact strings when they matter.** Required UI/API messages belong in quotes (e.g. "Enter a valid email address.") so tests and UI stay locked to the spec.
7. **Keep Given lean.** Only state what this scenario needs. Do not re-document the whole feature in every Given.
8. **One When (conceptually).** The action under test should be clear. Extra clicks that are part of the same action can be **And** under When; do not hide a second unrelated behavior in the same scenario.
9. **Align with FRs and the data model.** If FR-003 requires bcrypt and AC never mentions a stored hash (where relevant), something is missing. If AC mentions `dueDate` , the data model must include it.
10. **Write only what you can automate.** Every scenario will need at least one automated test before merge. If you cannot imagine a Jest/Vitest assertion, rewrite the Then.
11. **Stay inside the story.** US-2.1 scenarios are about creating lists — not completing todos (Feature 3) or editing profile (Feature 4).
12. **Use consistent actors and data.** Reuse stable examples (`jdoe` , list name "Groceries") across scenarios so readers can follow the narrative.

Template

```
## Acceptance Criteria

### US-N.1 – Short title (same as the story)

#### Scenario: Descriptive name (happy path)
* **Given** <starting state>
* **When** <action>
* **Then** <observable result>
* **And** <extra result if needed>

#### Scenario: Descriptive name (failure / edge)
* **Given** ...
* **When** ...
* **Then** ...
```

Given / When / Then tips

Step	Use for	Example
Given	Preconditions	"I am on the login page", "a user with username jdoe exists"
When	The action under test	"I click Sign in ", "I send POST /todo/login ..."
Then	Pass/fail outcome	"the API returns 401", "I am redirected to home"
And	Extra outcomes on the same scenario	"the error is displayed in a <v-alert type='error'>"

Do / don't (quick reference)

Do	Don't
Concrete examples with expected results	Abstract "user can authenticate successfully"
Separate happy path and error scenarios	One scenario that covers five outcomes

Exact messages and status codes when specified	"an error message appears" with no text
Scenario names unique within the feature	Duplicate or vague names (Test 1, Works)
Match ### US-N.n to the user story	Orphan scenarios with no story

Example

US-1.2 – Sign in

Scenario: User signs in with valid credentials

- * ****Given**** I am on the login page
- * ****And**** a registered user exists with username `jdoe` and a known password
- * ****When**** I enter username `jdoe` and the correct password
- * ****And**** I click ****Sign in****
- * ****Then**** the API returns `200` with a payload containing `userId`, `username`, `token`, and `role`
- * ****And**** I am redirected to the home page

Scenario: User signs in with invalid password

- * ****Given**** I am on the login page
- * ****And**** a registered user exists with username `jdoe`
- * ****When**** I enter username `jdoe` and an incorrect password
- * ****And**** I click ****Sign in****
- * ****Then**** the API returns `401` with `{ "message": "Invalid username or password." }`
- * ****And**** I remain on the login page

Coverage goal: every P1 story has at least one happy-path scenario **and** the important failure cases from Edge Cases / FRs.

7. Consistency checklist (before Status: Ready)

Use this before you fill in API / Screen polish sections.

- ☐ Header: Feature ID, branch pattern, Input, Depends on (if any)
- ☐ Every story has Priority, Independent test, and a matching ### US-N.n under Acceptance Criteria
- ☐ Story IDs match across User Stories and AC (US-2.3 ⇔ ### US-2.3 – ...)
- ☐ Every FR uses MUST / MUST NOT and is testable
- ☐ Key Entities list every concept this feature creates or changes
- ☐ Data Model Requirements tables cover new/changed fields (types + rules + FKs)
- ☐ Fields line up with FRs and Gherkin (no orphan columns; no missing required data)
- ☐ No unresolved [NEEDS CLARIFICATION: ...]
- ☐ Edge Cases appear as AC scenarios (or are explicitly deferred in Out of Scope later)
- ☐ Happy path + main failures covered for each P1 story
- ☐ Language is product-focused (what the user/system does), not a code walkthrough

8. What comes next

After the requirements portion is solid, write the **design portion** using [writing-feature-design.md](#):

- Data Ownership & Isolation
- API Requirements / Screen Requirements
- Refine Data Model Requirements + associations
- Test Coverage Map, Agent implementation request, Definition of Done, Out of Scope

Suggested draft order: Header → User Stories → FRs → Assumptions / Edge Cases / SC → **Key Entities + initial Data Model** → Gherkin AC → **then design sections** (see the design guide).

When unsure about process or IDs, follow [framework.md](#) and mirror [feature-1-user-auth.md](#).